

(1) Flex Program to display “Hello World”.

```
%{  
    #undef yywrap  
    #define yywrap() 1    //called by lex when input is exhausted  
                           //Return 1 if you are done or  
                           //0 if more processing is required  
%}  
  
%%  
  
[\\n] {  
    printf("Hello World\\n");  
}  
  
%%  
  
void main()  
{  
    yylex();    //calling the rules section  
}
```

(2) Flex program to check if the given letter is a vowel or not.

```
%{
    #undef yywrap
    #define yywrap() 1
    void display(int);

//If we enter any of the letters(a,e,i,o,u) then the first pattern with
//regular expression [a|e|i|o|u] matches; if anything other than those
//letters is given as an input, then the second action with regular
//expression(.+) matches

%}

%%

[a|e|i|o|u]] {
    int flag=1;
    display(flag);
    return;
}

.+ {
    int flag=0;
    display(flag);
    return;
}

%%
```

```
void display(int flag)
{
    if(flag==1)
        printf("The given letter [%s] is a vowel",yytext);
    else
        printf("The given letter [%s] is NOT a vowel",yytext);
    getch();
}
```

//yytext: extern char *yytext; is a global char pointer holding the
//currently matched lexeme

```
void main()
{
    printf("Enter a letter to check if it is a vowel or not: ");
    yylex();
}
```

(3) Flex program to check if the given string is a digit or a word.

```
%{  
    #undef yywrap  
    #define yywrap() 1  
    void display(char[],int);  
    int flag;  
  
}%  
  
%%  
  
[a-zA-Z]+ {  
  
    int flag=1;  
    display(yytext,flag);  
    return;  
}  
  
[0-9]+ {  
    int flag=0;  
    display(yytext,flag);  
    return;  
}  
  
.+ {  
    int flag=-1;  
    display(yytext,flag);  
    return;  
}
```

%%

```
void display(char a[],int flag)
{
    if(flag==1)
        printf("The given string = %s is a word ",a);
    if(flag==0)
        printf("The given string = %s is a number ",a);
    if(flag==-1)
        printf("The given string = %s is something else ",a);
    getch();
}
```

```
void main()
{
    printf("Enter a string: ");
    yylex();
}
```

(4) Flex program to implement positive closure.

```
%{
    #undef yywrap
    #define yywrap() 1
    //Check if input contains the string 11 or its combinations
}%

%%

"11"+ {
    printf("\nIt is a positive closure = ");
    ECHO;
}

.+ {
    printf("\nNot a positive closure = ");
    ECHO;
    getch();
    return;
}

%%

void main()
{
    yylex();
}

//ECHO copies yytext to the scanner's output
```

(5) Flex program to check well-formed parenthesis (bracket matching).

```
%{
    #undef yywrap
    #define yywrap() 1
    int flag=0;

%}

%%

"(" { flag++;
}

")" { flag--;
}

[\n] {
    if(flag==0)
        printf("\n The statement has no missing parenthesis\n\n");
    else
        printf("\n Error \n");
    if(flag>0)
        printf("The statement has either extra ( parenthesis or a missing )
        parenthesis\n\n");
    else if(flag<0)
        printf("The statement has either extra ) parenthesis or a missing
        ( parenthesis\n\n");
    flag=0;
}
```

```
%%
```

```
void main()  
{  
    yylex();  
}
```


(6) Flex program to implement a calculator.

```
%{
```

```
    #undef yywrap
    #define yywrap() 1
    int f1=0,f2=0;
    char oper;
    float op1=0,op2=0,ans=0;
    void eval();
```

```
%}
```

```
DIGIT [0-9]
```

```
NUM {DIGIT}+(\.{DIGIT})?
```

```
OP [*/+]
```

```
%%
```

```
{NUM} {
```

```
    if(f1==0)
```

```
    {
```

```
        op1=atof(yytext);
```

```
        // atof() converts the string argument to a floating point number
```

```
        f1=1;
```

```
    }
```

```
    else if(f2==1)
```

```
    {
```

```
        op2=atof(yytext);
```

```
        f2=1;
    }

    if((f1==1) && (f2==1))
    {
        eval();
        f1=0;
        f2=0;
    }
}
```

```
{OP} {
    oper=(char) *yytext;
    f2=-1;
}
```

```
[\n] {

    if(f1==1 && f2==1)
    {
        eval();
        f1=0;
        f2=0;
    }
}
```

```
%%
```

```
void main()
{
    yylex();
}
```

```
void eval()
{
    switch(oper)
    {
        case '+':
            ans=op1+op2;
            break;

        case '-':
            ans=op1-op2;
            break;

        case '*':
            ans=op1*op2;
            break;

        case '/':
            if(op2==0)
            {
                printf("ERROR");
                return;
            }
            else
            {
```

```
        ans=op1/op2;
    }
    break;
default:
    printf("Operation not available");
    break;
}
printf("The answer is = %lf",ans);
}
```

(7) Flex program to count the number of lines, tabs and spaces used in the input.

```
%{
    #undef yywrap
    #define yywrap() 1
    int nlines,nwords,nchars;
}%

%%
    [\n] { nchars++;nlines++; }
    [^ \n\t]+ { nwords++, nchars=nchars+yyleng; }
    . { nchars++; }
%%

void main()
{
    yylex();
    printf("Lines = %d\nChars = %d\nWords = %d",nlines,nchars,nwords);
}

//yyleng: extern int yylen; is a global int that contains the length of the
//currently matched lexeme
```

(8) Bison program to evaluate a postfix expression (Reverse Polish Notation).

```
/* Reverse Polish Notation calculator. */

%{
    #include <stdio.h>
    #include <stdlib.h>
    #include <ctype.h>
    #include <math.h>
    int yylex (void);
    void yyerror (char const *);
}%

#define api.value.type {double}
%token NUM

%% /* Grammar rules and actions follow. */

input:
    %empty
    | input line
    ;

line:
    '\n'
    | exp '\n'    { printf ("%10g\n", $1); }
    ;

exp:
```

```

NUM
| exp exp '+' { $$ = $1 + $2; }
| exp exp '-' { $$ = $1 - $2; }
| exp exp '*' { $$ = $1 * $2; }
| exp exp '/' { $$ = $1 / $2; }
| exp exp '^' { $$ = pow ($1, $2); } /* Exponentiation */
| exp 'n' { $$ = -$1; } /* Unary minus */
;
%%

```

```

/* The lexical analyzer */

```

```

int yylex (void)
{
    int c = getchar ();
    /* Skip white space. */
    while (c == ' ' || c == '\t')
        c = getchar ();
    /* Process numbers. */
    if (c == '.' || isdigit (c))
    {
        ungetc (c, stdin);
        if (scanf ("%lf", &yylval) != 1)
            abort ();
        return NUM;
    }
    /* Return end-of-input. */
    else if (c == EOF)
        return YYEOF;
    /* Return a single char. */
}

```

```
else  
    return c;  
}
```

```
int main ()  
{  
    return yyparse ();  
}
```

```
void yyerror (char const *s)  
{  
    fprintf (stderr, "%s\n", s);  
}
```